

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

```
Windows PowerShell
option to transmit
-4, --ipv4          use IPv4 addresses only
-6, --ipv6          use IPv6 addresses only
--[no-]filter <args> object filtering
--[no-]also-filter-submodules
                    apply partial clone filters to submodules
--[no-]remote-submodules
                    any cloned submodules will use their remote-tracking branch
--[no-]sparse       initialize sparse-checkout file to include only files at root
--[no-]bundle-uri <uri>
                    a URI for downloading bundles before fetching from origin remote

PS C:\Users\kevin\IT314> git clone https://github.com/Kebs-deck/IT314AB_Villacorte.git
Cloning into 'IT314AB_Villacorte'...
remote: Enumerating objects: 256, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 256 (delta 3), reused 8 (delta 3), pack-reused 247 (from 2)
Receiving objects: 100% (256/256), 41.61 MiB | 825.00 KiB/s, done.
Resolving deltas: 100% (52/52), done.
PS C:\Users\kevin\IT314> cd C:\Users\kevin\IT314\IT314AB_Villacorte
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git branch
* main
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git checkout feature/student-list
branch 'feature/student-list' set up to track 'origin/feature/student-list'.
Switched to a new branch 'feature/student-list'
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git branch
* feature/student-list
  main
PS C:\Users\kevin\IT314\IT314AB_Villacorte>
```

- Terminal showing your current branch

Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart **List**.

Research what a Dart **List** is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

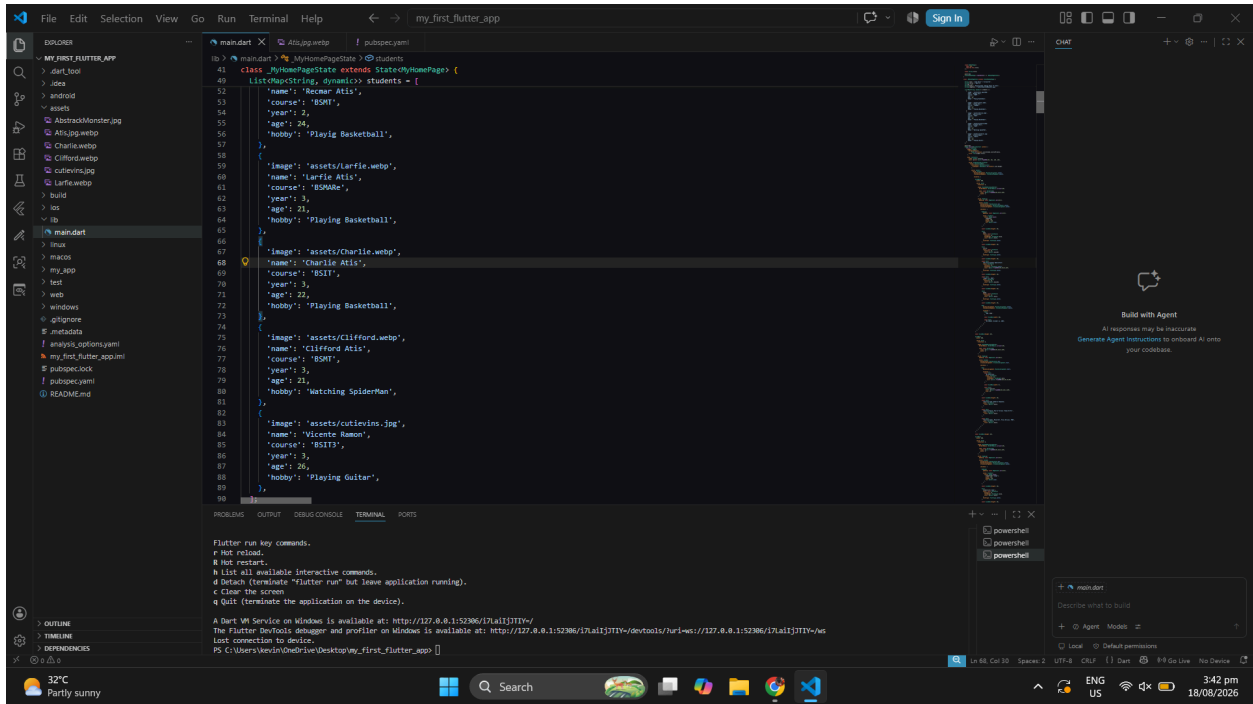
Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:



- Your student list
- At least five student entries

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

```

Padding(
  padding: const EdgeInsets.all(15.0),

  child: ClipOval(
    child: Image.asset(
      'assets/Atis.jpg.webp',
      width: 110,
      height: 110,
      fit: BoxFit.cover,
    ), // Image.asset
  ), // ClipOval
), // Padding

const SizedBox(height: 20),

Text(
  name2 ?? 'Name: Not provided',
  style: const TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.black,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

Text(
  course2 ?? 'Course: Unknown',
  style: const TextStyle(
    fontSize: 20,
    color: Colors.black87,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

```

```

Padding(
  padding: const EdgeInsets.all(15.0),

  child: ClipOval(
    child: Image.asset(
      'assets/Larfile.webp',
      width: 110,
      height: 110,
      fit: BoxFit.cover,
    ), // Image.asset
  ), // ClipOval
), // Padding

const SizedBox(height: 20),

Text(
  name3 ?? 'Name: Not provided',
  style: const TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.black,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

Text(
  course3 ?? 'Course: Unknown',
  style: const TextStyle(
    fontSize: 20,
    color: Colors.black87,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

```

```

children: [

  Padding(
    padding: const EdgeInsets.all(15.0),

    child: ClipOval(
      child: Image.asset(
        'assets/Charlie.webp',
        width: 110,
        height: 110,
        fit: BoxFit.cover,
      ), // Image.asset
    ), // ClipOval
  ), // Padding

  const SizedBox(height: 20),

  Text(
    name4 ?? 'Name: Not provided',
    style: const TextStyle(
      fontSize: 24,
      fontWeight: FontWeight.bold,
      color: Colors.black,
    ), // TextStyle
    textAlign: TextAlign.center,
  ), // Text

  const SizedBox(height: 10),

  Text(
    course4 ?? 'Course: Unknown',
    style: const TextStyle(
      fontSize: 20,
      color: Colors.black87,
    ), // TextStyle
    textAlign: TextAlign.center,
  ), // Text

```

```

const Text(
  'Clifford Atis',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.black,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

const Text(
  'BSIT-3',
  style: TextStyle(
    fontSize: 20,
    color: Colors.black87,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

const Text(
  'August 13, 2026',
  style: TextStyle(
    fontSize: 18,
    color: Colors.black54,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 25),

const Text(
  'Watching SpiderMan',
  style: TextStyle(

```

```
const Text(
  'Vicente Ramon',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.black,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

const Text(
  'BSIT-3',
  style: TextStyle(
    fontSize: 20,
    color: Colors.black87,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

const Text(
  'August 13, 2026',
  style: TextStyle(
    fontSize: 18,
    color: Colors.black54,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 25),

const Text(
  'Playing Guitar',
  style: TextStyle(
    fontSize: 15,
```

- Your five manually created student cards (do not change your past code yet)

🚩 Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

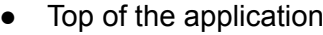
Your application should now allow continuous vertical scrolling.

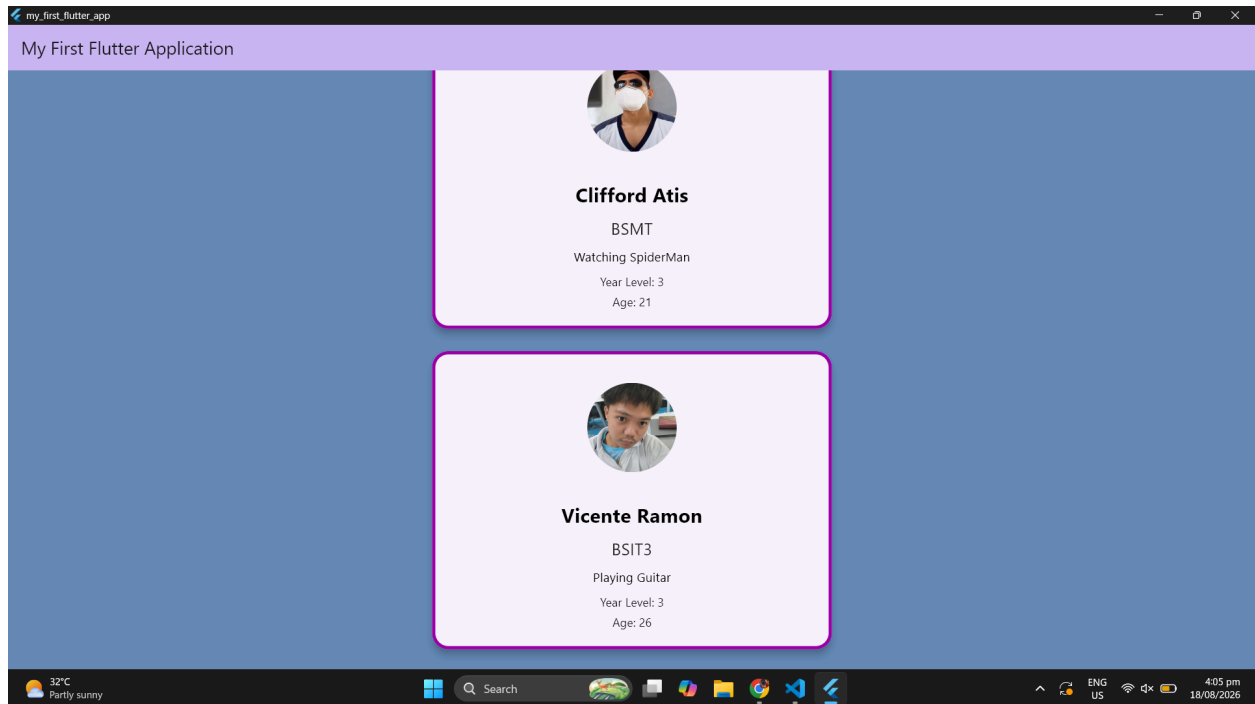
Test

The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:





- Bottom portion showing later students

🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

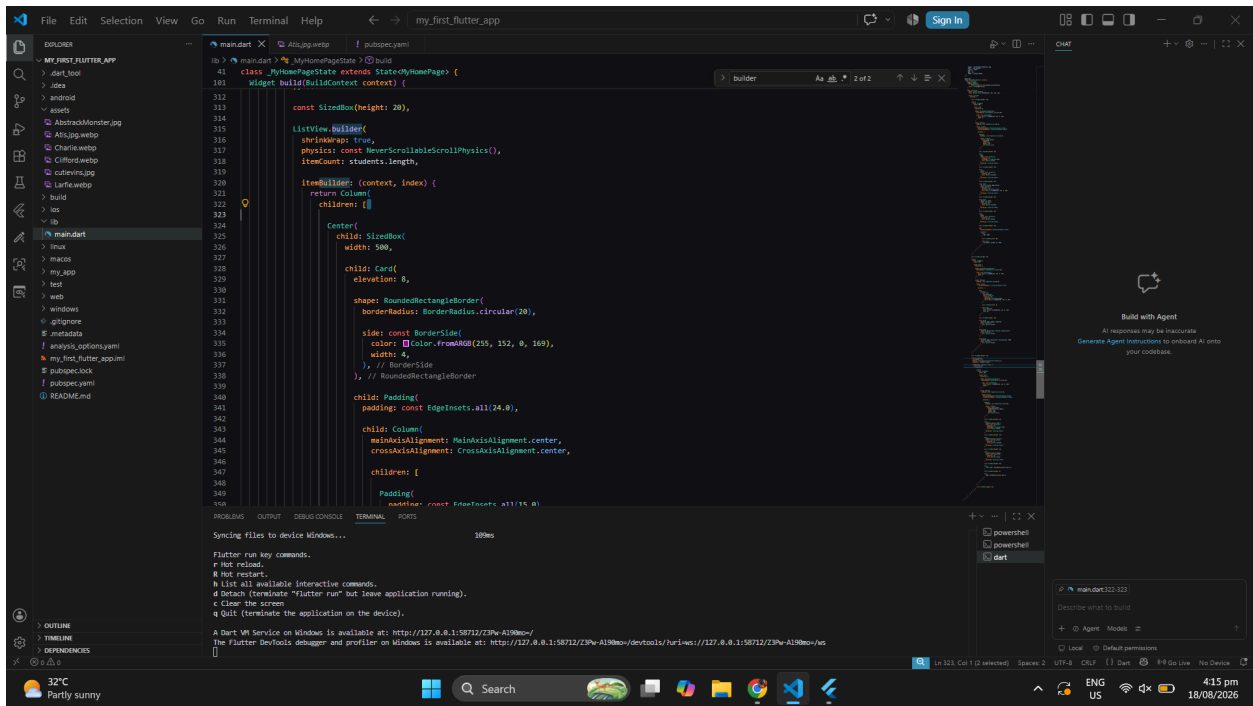
Your UI should now create one card for each student in your list.

Test

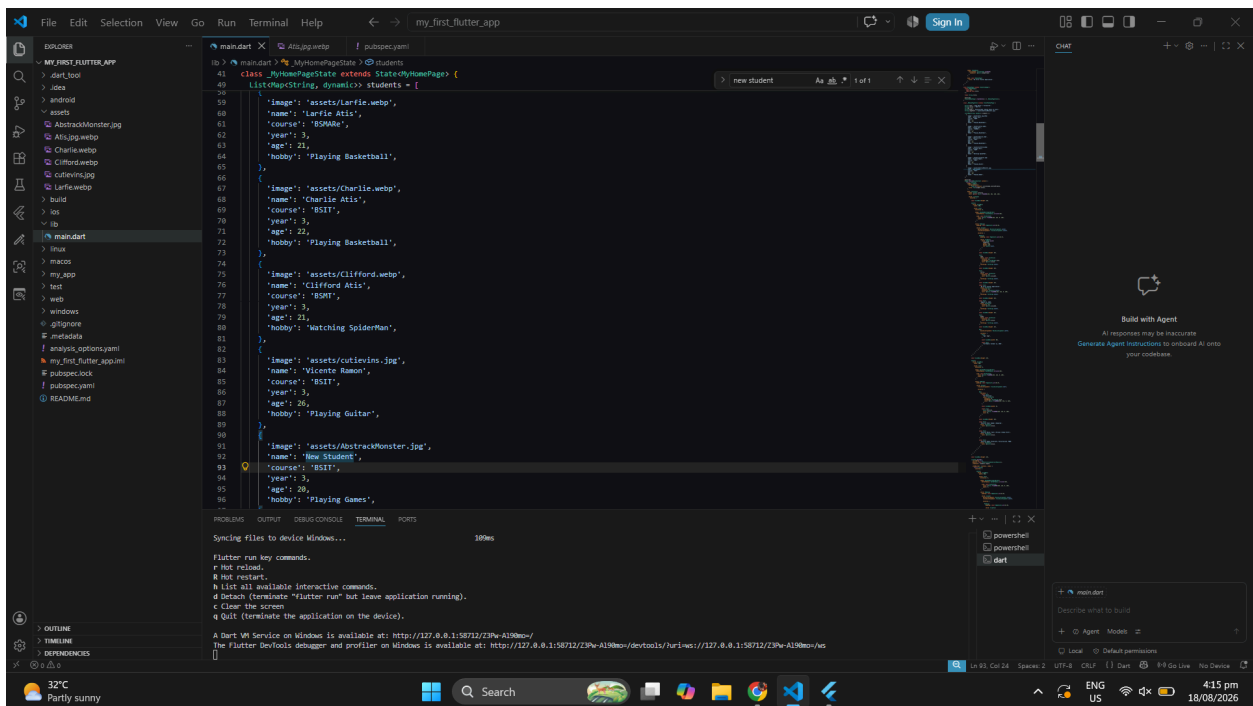
Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

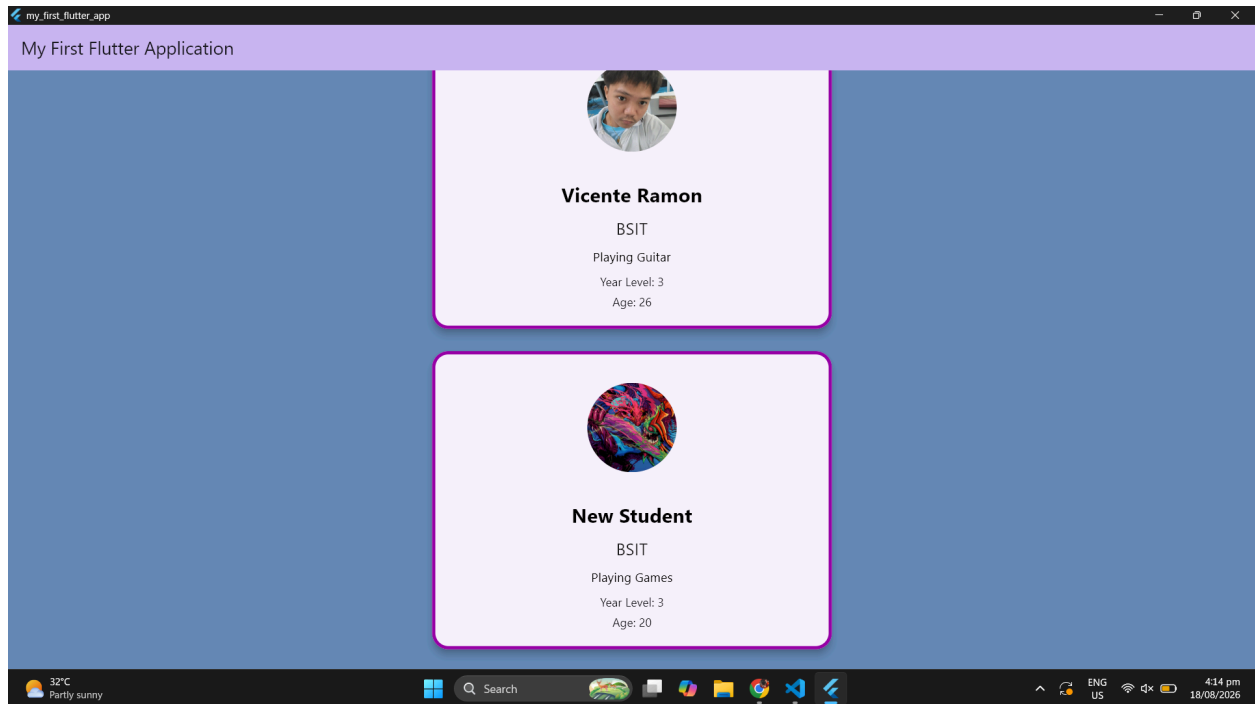
Screenshot:



- **ListView.builder** in your code



- **Student list**



- Newly generated student appearing in the app

🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

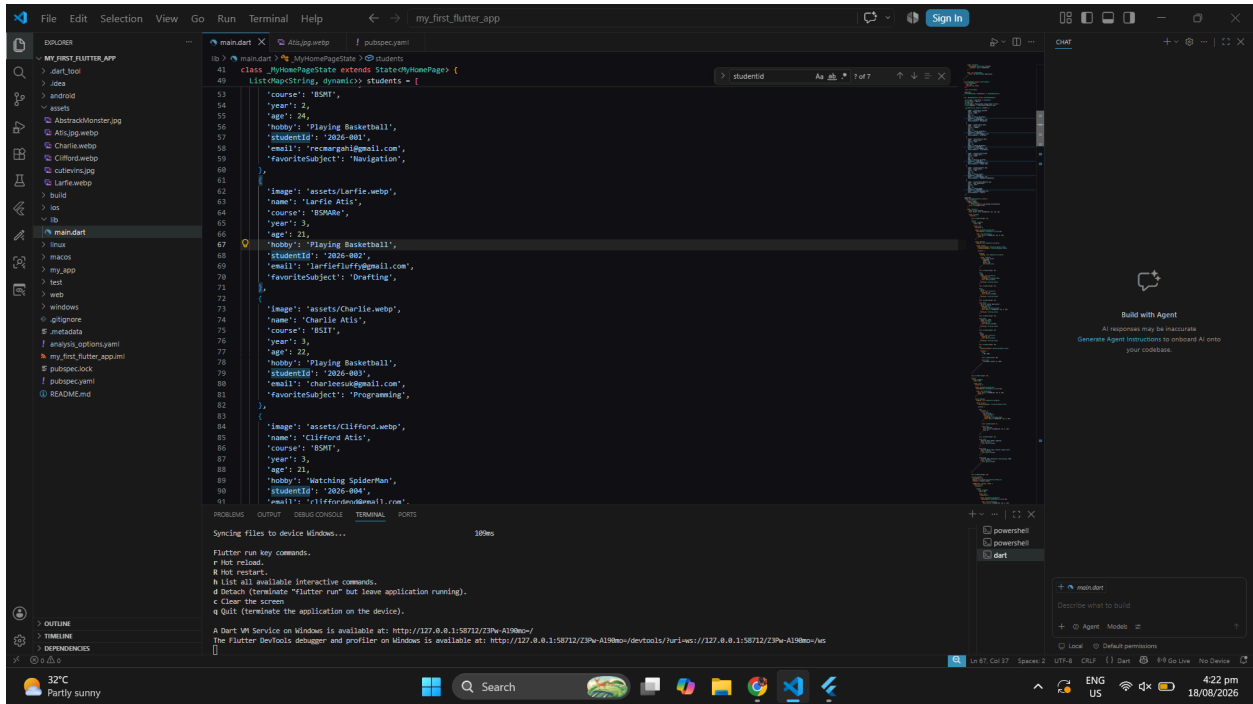
Add at least three new fields to every student.

Examples:

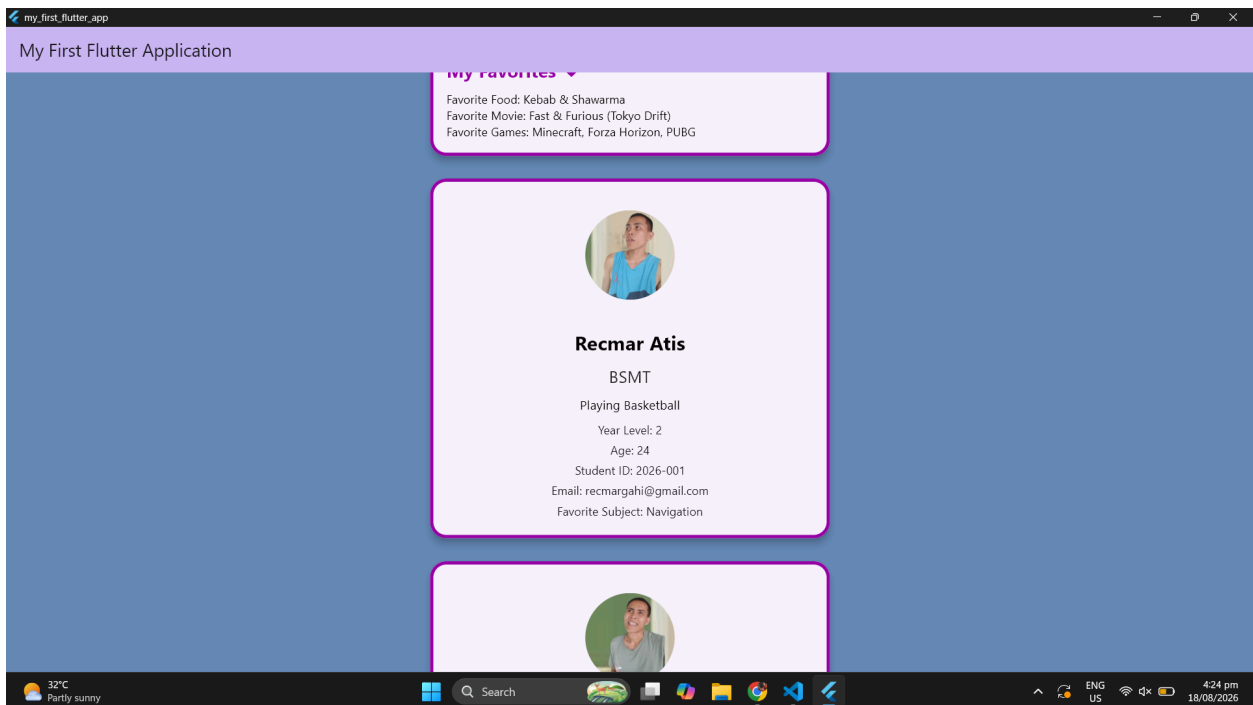
- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:



- Updated student data



- Student Card displaying the additional fields

🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

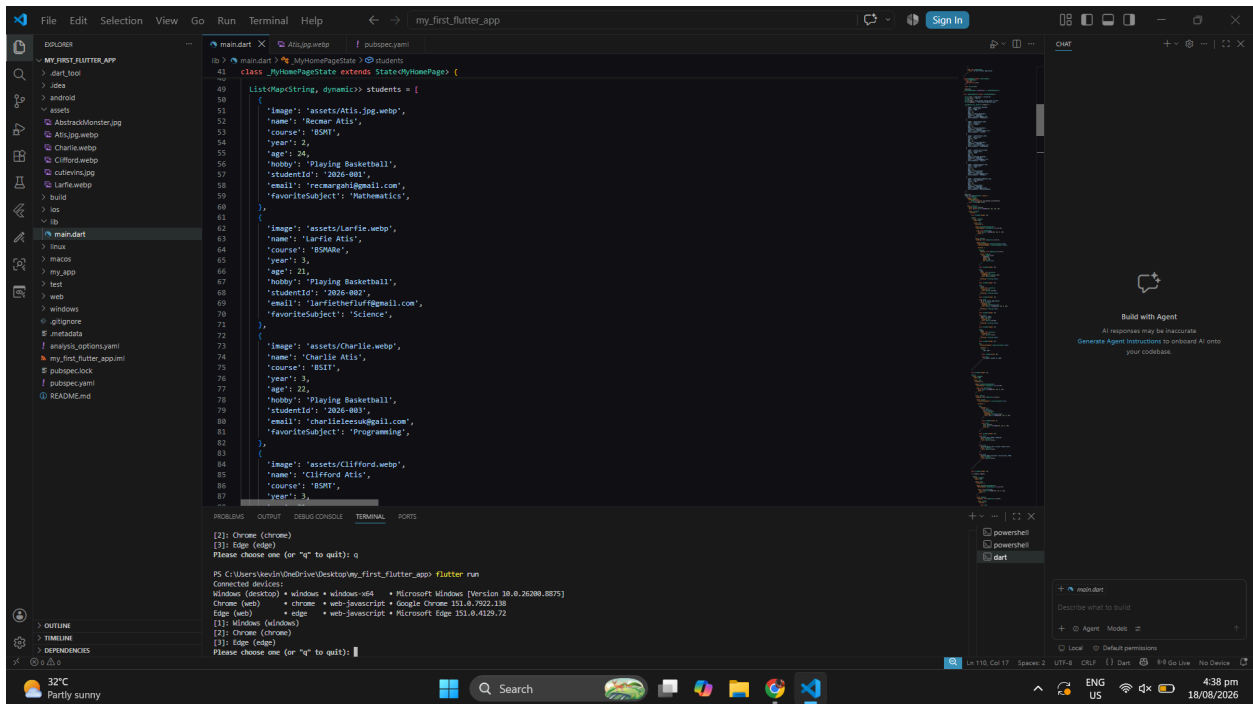
Examples:

- "No students found."
- "Student list is empty."

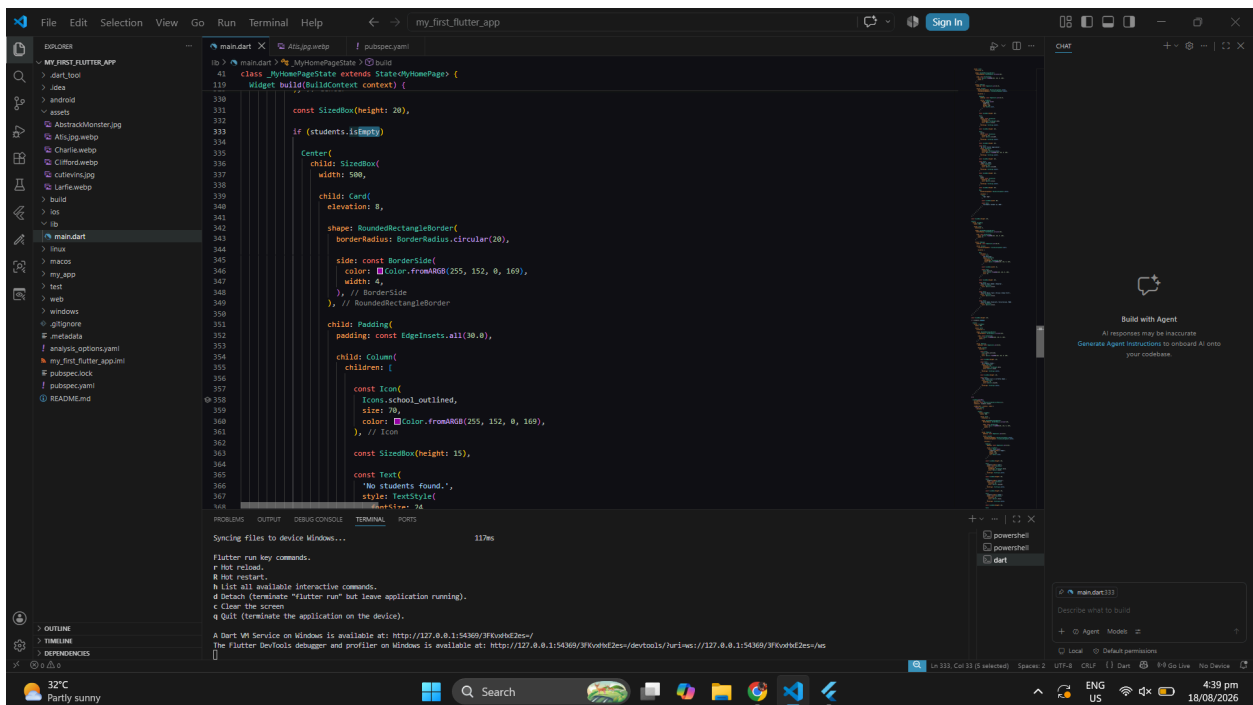
Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

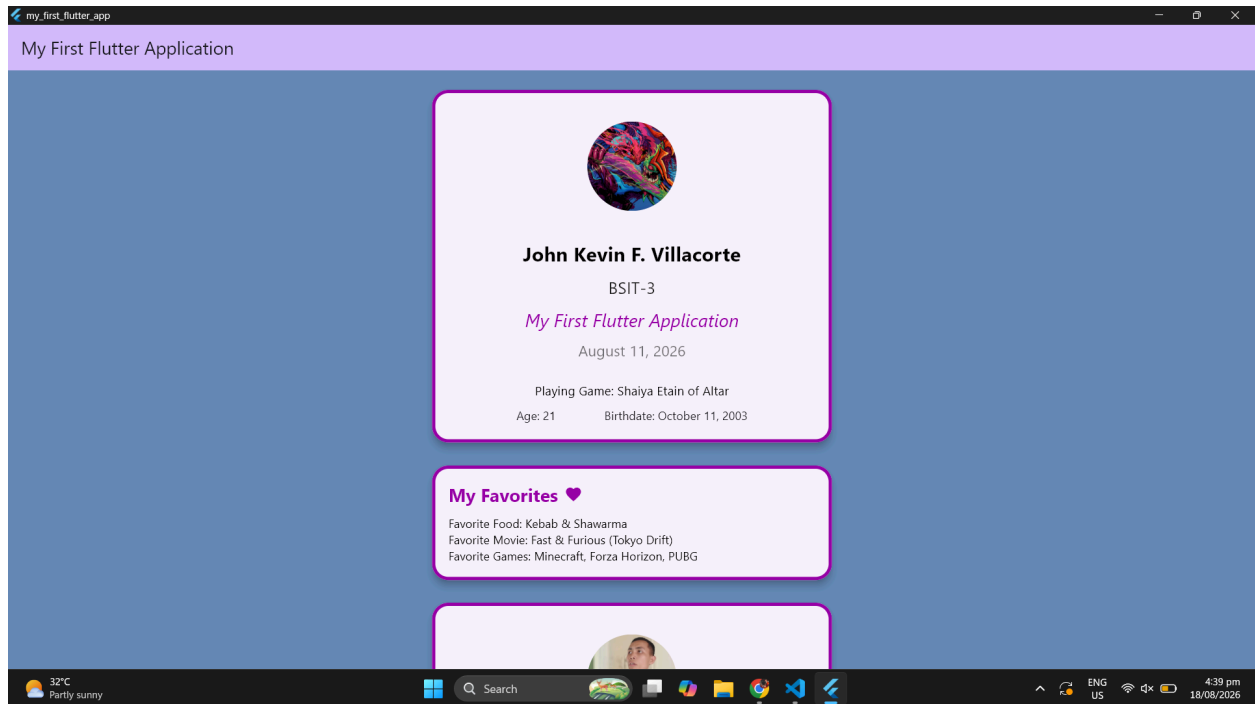
Screenshot:



- Empty student list



- Empty-state UI



- Application still functioning correctly

🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

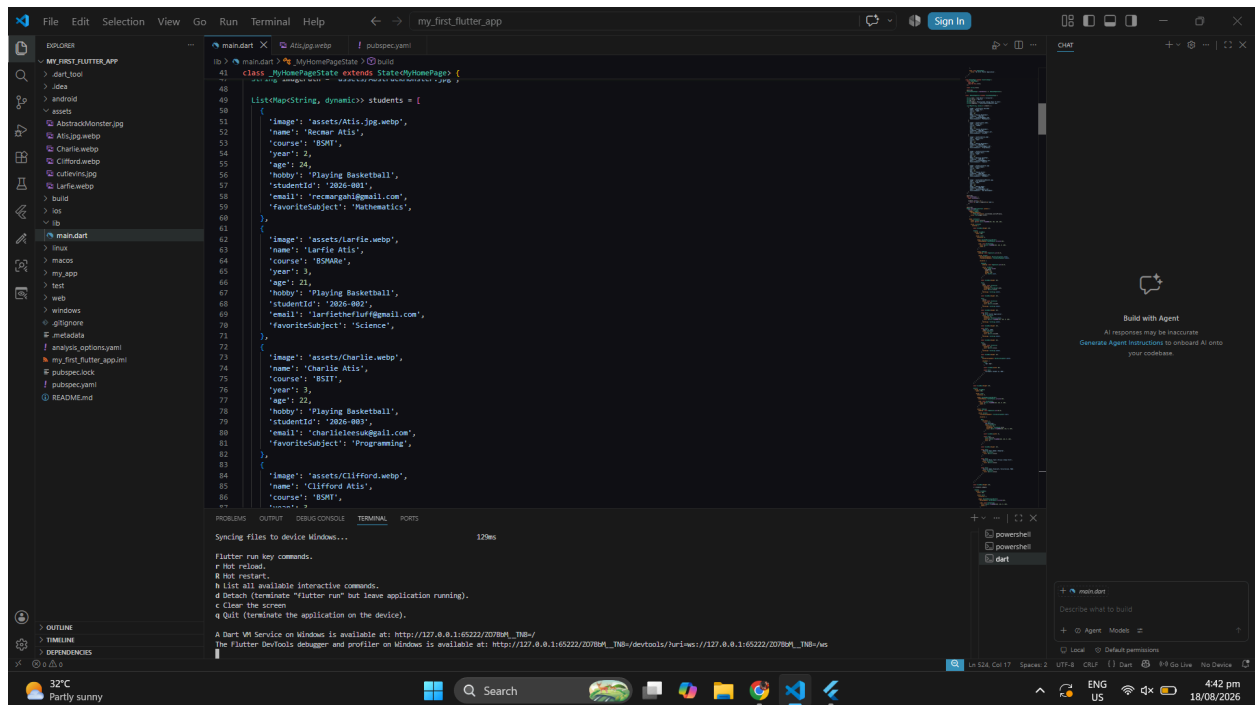
After sorting:

- Aaron

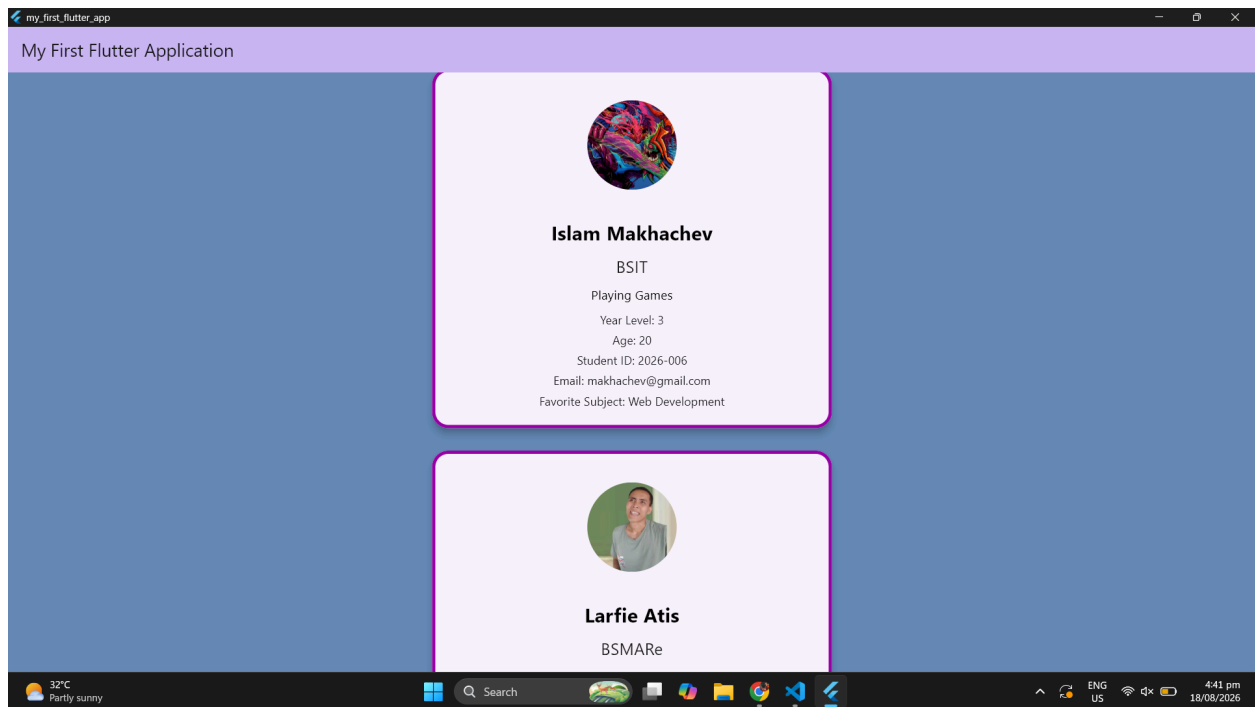
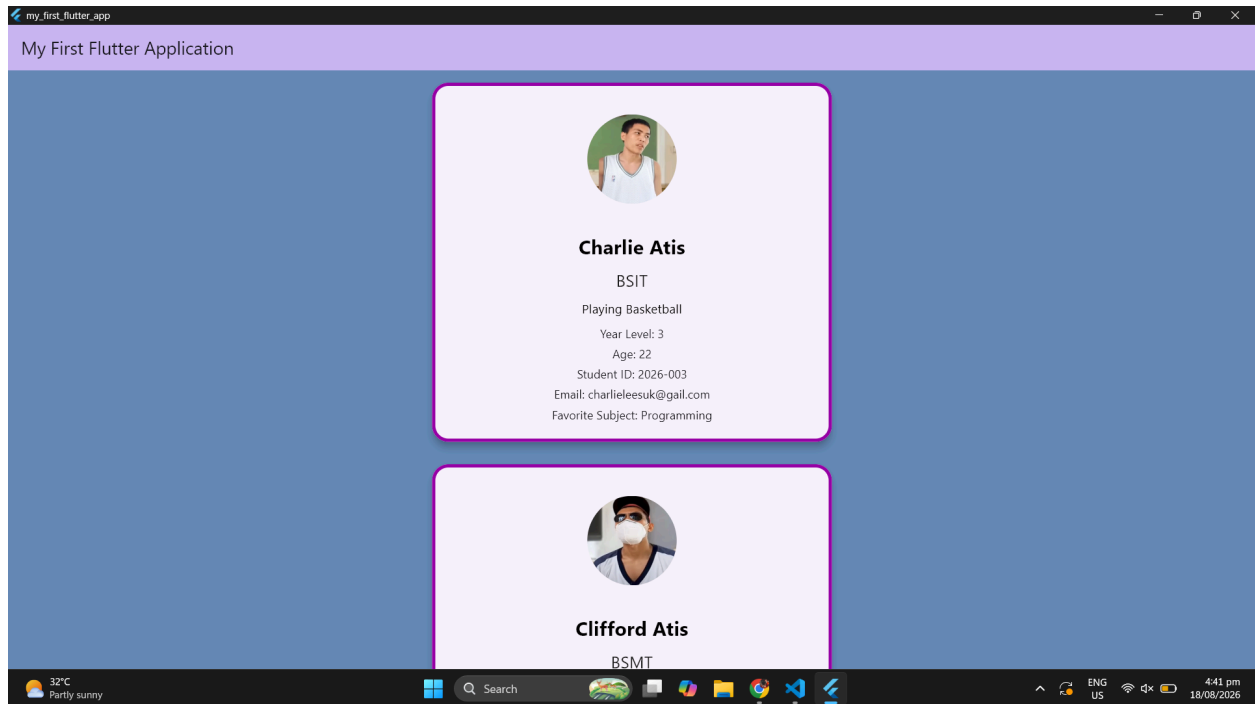
- John
- Maria
- Sophia

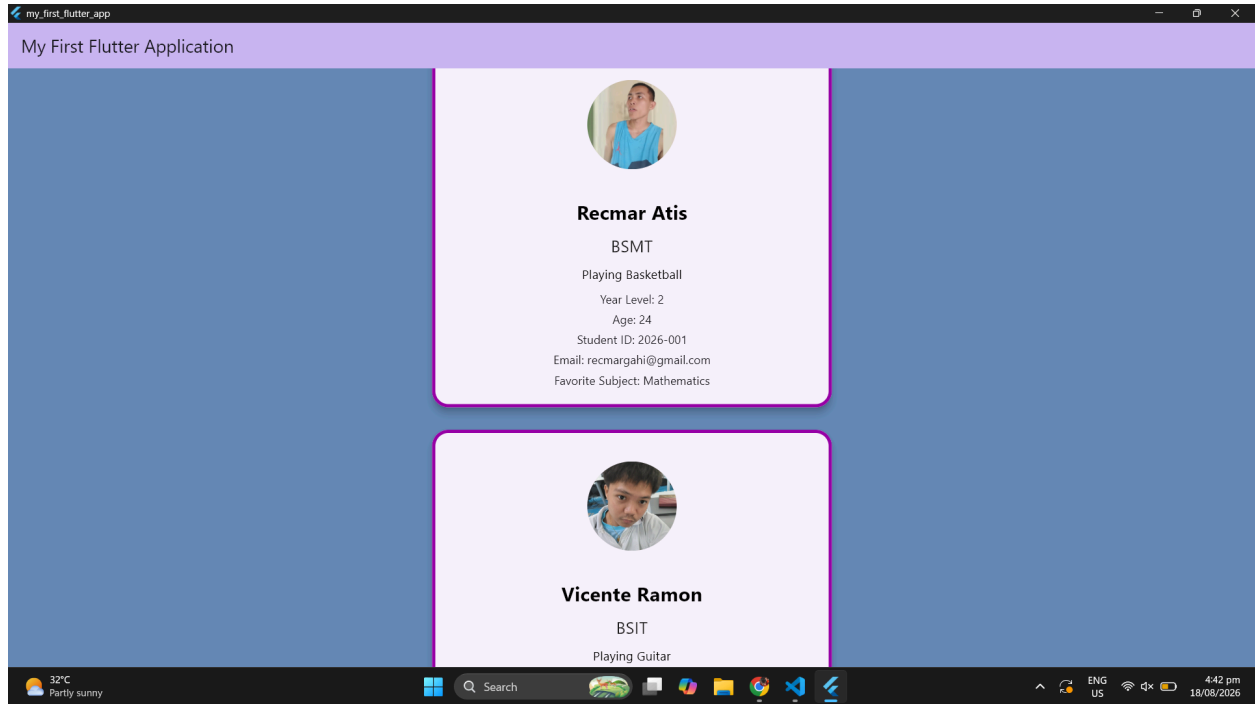
Your UI should immediately reflect the new order.

Screenshot:



- Student list before or after sorting





- Application displaying the sorted order

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit